



Department of Electrical and Information Engineering

University of Ruhuna

Mini Project Proposal

Image Processing and Computer Vision

EC7205

Arachchi WATTW - EG/2021/4412

Lareef J.M. - EG/2021/4644

Nadha MJ - EG/2021/4686

Safeer SM - EG/2021/4771

Submission date - 21.01.2026

Department of Electrical and Information Engineering
University of Ruhuna

Contents

Contents	1
1 Problem Statement	2
2 Objectives	3
3 Literature Review	4
3.1 Study 1: Jayanth et al. (2023) – Traditional Ball Detection using Python [2]	4
3.2 Study 2: Shah and Khalil (2018) – Centroid and Contour-Based Tracking [1]	4
3.3 Study 3: Redmon et al. (2016) – You Only Look Once (YOLO)	5
3.4 Study 4: IJARCCCE (2025) – AI-Based Cricket Umpiring and Trajectory Prediction [6]	5
4 Methodology	7
4.1 Software Frameworks and Development Environment	7
4.2 Image Acquisition and Pre-processing	8
4.3 Segmentation via Color Masking	8
4.4 Feature Extraction and Contour Analysis	8
4.5 Tracking via Centroid Calculation	8
4.6 Trajectory Visualization and Regression	9
5 Dataset / Data Collection	10
6 Expected Outcomes	11
7 Percentage of Image Processing & Computer Vision Involvement	12
8 References	13

1. Problem Statement

In modern cricket, analyzing ball movements such as swing, seam, and flight path is essential for coaching and player development. However, manually tracking these details is an exhausting and time-consuming task for analysts. While professional technologies like Hawk-Eye provide high accuracy, they are extremely expensive and require complex multi-camera setups that are inaccessible to local clubs and schools [1].

There is a significant need for a lightweight, automated solution that can detect a cricket ball and visualize its trajectory using standard video footage, such as smartphone recordings [10]. Although deep learning offers high precision, it often requires high-end hardware and massive labeled datasets [2]. This project addresses the limitations of these resource-intensive systems by developing an efficient, real-time visualization tool. By using traditional OpenCV techniques, specifically **HSV color masking** and **centroid tracking**, we aim to provide a practical, low-cost alternative for sports motion analysis [10]. [Back to Table of Contents](#)

2. Objectives

The primary goal of this mini-project is to develop a functional and accessible tool for cricket analytics using traditional computer vision. The specific objectives are as follows:

- Build a system that can take a standard cricket video and break it down into individual frames for detailed analysis.
- Create a reliable detection method using **HSV color masking** to identify the ball from complex stadium backgrounds and varying lighting conditions.
- Apply noise-reduction techniques such as **Gaussian blurring** and **morphological transformations** to minimize distractions caused by players' clothing and stadium lighting.
- Use **centroid calculation** to determine the exact center of the ball in each frame and record these coordinates to map its movement over time.
- Demonstrate the ball's flight path by drawing a continuous and smooth trajectory on the video, similar to professional broadcasting tools.
- Evaluate system performance by measuring processing speed (frames per second) and tracking accuracy from the bowler's hand to the pitch.

[Back to Table of Contents](#)

3. Literature Review

The field of sports analytics, particularly ball tracking, has evolved significantly over the past few decades. Early research relied on basic pixel-level image manipulation, whereas modern approaches increasingly employ complex deep learning architectures. This section reviews influential studies that inform the methodology adopted in this project.

3.1. Study 1: Jayanth et al. (2023) – Traditional Ball Detection using Python [2]

This study presents a practical framework for tracking spherical objects using the OpenCV library in Python. The researchers employed a two-stage approach: first, detecting the circular shape of the ball using the Hough Circle Transform, and second, tracking its motion using optical flow.

- The system was successfully tested on low-power embedded platforms such as the Raspberry Pi, demonstrating that real-time tracking is achievable without expensive GPUs by relying on geometric and color-based features.
- The primary strength of the approach is its high processing speed and low computational cost. However, a key limitation is its sensitivity to circular noise, where other round objects (e.g., a player's head or stadium clocks) may be falsely detected as the ball.
- Our project adopts the Python/OpenCV framework proposed in this study but enhances robustness by incorporating **HSV color masking** to ensure that only objects of a specific color are tracked.

3.2. Study 2: Shah and Khalil (2018) – Centroid and Contour-Based Tracking [1]

This paper investigates ball detection techniques for robotic applications, emphasizing the use of the ball's centroid as a distance and position measure. Color intensity thresholding is applied to isolate the ball, followed by contour analysis to compute its center.

- By calculating centroid coordinates (x, y) using image moments, the system accurately estimated the physical distance between the ball and the camera, enabling precise robotic navigation.
- The study provides strong mathematical insight into centroid-based tracking. However, it was designed for a tennis ball in a controlled environment.
- In a cricket stadium, where background clutter is significantly higher, this approach would require additional preprocessing to remain effective.
- The centroid calculation logic from this study is directly applied in our project to record the coordinates required for trajectory visualization.

3.3. Study 3: Redmon et al. (2016) – You Only Look Once (YOLO)

YOLO introduced a unified approach to object detection in which a single neural network predicts bounding boxes and class probabilities in one pass.

- YOLO achieved real-time detection speeds of up to 45 frames per second while maintaining high accuracy, making it a standard solution in professional sports broadcasting.
- The major strength of YOLO lies in its ability to distinguish objects such as balls and players with high confidence.
- However, its limitations include the black-box nature of deep learning models, the requirement for high-end hardware, and the need for thousands of labeled training images.
- In contrast, this project proposes a non-deep-learning alternative, demonstrating that for a single high-contrast object like a cricket ball, traditional image processing can yield comparable visual results with significantly lower computational cost.

3.4. Study 4: IJARCCCE (2025) – AI-Based Cricket Umpiring and Trajectory Prediction [6]

This study explores the use of artificial intelligence to assist cricket umpires in Leg Before Wicket (LBW) decisions by predicting the future trajectory of the ball.

- Polynomial regression is applied to detected ball coordinates to estimate the expected flight path beyond the point of impact.
- While highly effective in professional settings, the system relies on high-quality camera feeds.
- Low-resolution smartphone or online video footage often introduces motion blur, reducing the accuracy of the regression model.
- Our project implements a simplified version of this trajectory drawing logic to visually represent the ball's motion.

Gap Analysis and Project Contribution

Existing research can be broadly categorized into two extremes:

1. **Simple Detection:** Studies such as [1] focus on ball detection but lack professional-grade trajectory visualization.

2. **Heavy Deep Learning:** Approaches such as YOLO [2] deliver high accuracy but demand large datasets and expensive GPU hardware, making them impractical for students and local cricket clubs.

Our Project's Contribution - Our project fills the gap between these two extremes. We are developing a Medium-Complexity solution. By combining the lightweight nature of traditional OpenCV techniques (HSV masking and Centroids) with a professional-grade visualization output (Trajectory Trails), we are creating a tool that is both scientifically transparent and practically useful. Unlike deep learning models, our system doesn't need a training phase it works as soon as you define the ball's color, making it a perfect tool for immediate use in local cricket coaching.

[Back to Table of Contents](#)

4. Methodology

The methodology for this project centers on a robust, real-time image processing pipeline designed to operate on standard consumer-grade hardware. While modern trends favor deep learning, this project utilizes Traditional Computer Vision techniques to ensure transparency, high execution speed, and minimal hardware requirements. The entire system is built using the OpenCV (Open Source Computer Vision) library within a Python programming environment, leveraging its highly optimized functions for geometric and color-based analysis. The overall system architecture of the proposed solution is illustrated in Figure 1.

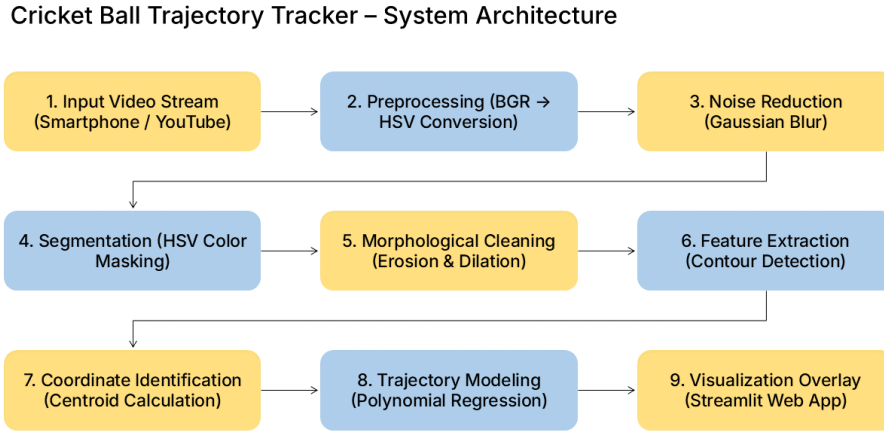


Figure 1: Cricket Ball Trajectory Tracker – System Architecture

4.1. Software Frameworks and Development Environment

The software architecture is modular, allowing for independent testing of detection and tracking components.

- **Python** - Selected as the primary language due to its extensive ecosystem for scientific computing and its user-friendly nature for integrating various libraries.
- **OpenCV** - This is the core engine of the project. It provides the necessary tools for frame-by-frame video decoding, color space transformations, and geometric feature extraction.
- **NumPy** - Used for fast numerical array manipulations, particularly when defining the color thresholding ranges (the Numpy arrays for upper and lower color bounds).
- **Streamlit** - This framework will be used to deploy the final tool as a web application, providing an interface for users to upload videos and view real-time trajectory overlays without needing local code execution.

4.2. Image Acquisition and Pre-processing

The process begins with Video Capture, where the system ingests standard video frames (from a smartphone or YouTube) at a fixed frame rate. Before detection occurs, the raw data must be cleaned to ensure the ball is the most prominent feature:

1. **Color Space Conversion** - Digital video is typically processed in BGR. To isolate a specific color like a red or green cricket ball, we convert frames to HSV (Hue, Saturation, Value). The HSV space separates color (Hue) from brightness (Value), making the system resilient to stadium shadows or changing sunlight.
2. **Noise Reduction (Gaussian Blur)** - A Gaussian Blur is applied to the frames. This mathematical smoothing reduces salt and pepper noise and ensures that small, bright background objects do not interfere with the detection of the ball's edges.

4.3. Segmentation via Color Masking

Segmentation is the process of separating the ball from the complex stadium background.

- **Thresholding** - The system defines a range of acceptable color values (e.g., the specific red of a cricket ball). It creates a binary mask where pixels matching the ball's color are turned white and all other pixels are black.
- **Morphological Transformations** - To further refine this mask, Erosion and Dilation are applied. Erosion removes tiny white speckles that might be noise, while Dilation fills in internal gaps within the ball's shape, ensuring a solid blob to track.

4.4. Feature Extraction and Contour Analysis

With a clean binary mask, the system identifies which blob is actually the ball.

- **Finding Contours** - The algorithm uses the Contour Method to detect the boundaries of all objects in the mask.
- **Geometric Filtering** - Since a stadium might have other red objects, the system filters contours based on circularity and area. By detecting the circular shape and calculating the minimum enclosing circle, the system discards straight segments or objects too large or too small to be a ball.

4.5. Tracking via Centroid Calculation

Once the ball is identified in a single frame, its position is recorded to create a trajectory.

- **Image Moments** - The centroid (geometric center) of the ball's contour is calculated using mathematical moments in the X and Y directions. This provides a single (x, y) coordinate for each frame.

- **Coordinate Correspondence** - The system maintains a memory of past centroids (using a deque or list). For every new frame, it checks if the new centroid exists and matches the physical characteristics (velocity and size) of the previous one, ensuring the tracker doesn't jump to a different object.

4.6. Trajectory Visualization and Regression

The final and most visual phase is turning coordinates into a Hawk-Eye trail.

1. **Curve Fitting** - Detection might have slight errors or jitter, so Polynomial Regression (quadratic curve, $y = ax^2 + bx + c$) is applied. This creates a smooth, mathematically accurate path representing the ball's movement.
2. **Persistent Overlay** - Using OpenCV drawing functions, the system overlays the calculated trajectory onto the original video frame in real-time. This creates a trail effect where the viewer can see the entire history of the ball's path simultaneously.

[Back to Table of Contents](#)

5. Dataset / Data Collection

This project utilizes the Cricket Ball Dataset sourced from Kaggle, comprising 1,778 high-resolution images in YOLOv8 format.

Dataset Link: <https://www.kaggle.com/datasets/kushagra3204/cricket-ball-dataset-for>

These images feature diverse characteristics, including red and white balls under varying stadium lighting conditions (day and night) and complex backgrounds such as practice nets and cheering crowds. While the original dataset includes annotations, this project focuses on the raw image data to refine HSV color-masking algorithms.

To supplement the dataset, we recorded 10 custom video clips using a smartphone mounted on a tripod. This ensures high-quality, steady footage for trajectory testing, maintaining ethical standards by recording only consenting participants in public or local club settings.

[Back to Table of Contents](#)

6. Expected Outcomes

The successful completion of this project will result in a practical demonstration of traditional computer vision's effectiveness in sports analytics. The expected outcomes are as follows:

- **Functional Web Application:** A deployed Streamlit-based interface allowing users to upload cricket footage and receive immediate trajectory analysis.
- **Source Code and Documentation:** A well-documented Python repository featuring the OpenCV pipeline, including HSV masking and centroid tracking scripts.
- **Real-Time Processing Speed:** Achievement of a processing rate of at least 30 frames per second (FPS) on standard CPU hardware, demonstrating the efficiency of non-deep learning methods.
- **High Detection Accuracy:** A target accuracy of 85–90% in identifying the ball's centroid across various frames, mirroring the success rates found in established academic research.
- **Trajectory Visualization:** A clear, visual "trail" overlay on the video that effectively proves the system's ability to map continuous motion from discrete coordinate data.

[Back to Table of Contents](#)

7. Percentage of Image Processing & Computer Vision Involvement

This project is strictly designed to be a computer vision-centric application, with approximately 90% of the core logic dedicated to image processing and vision techniques. The project satisfies this requirement through the following breakdown:

- **Preprocessing (20%):** Every frame undergoes conversion from BGR to the HSV color space and the application of Gaussian blurring to eliminate environmental noise.
- **Segmentation (30%):** We implement HSV-based thresholding and morphological operations (Erosion and Dilation) to isolate the ball from complex stadium backgrounds.
- **Feature Extraction & Tracking (40%):** The system uses Contour Detection and Centroid Calculation based on image moments to identify the ball's (x, y) coordinates in real-time.
- **Mathematical Modeling (10%):** We utilize Polynomial Regression to smooth the detected points and visualize a continuous trajectory trail.

All components will be developed using the OpenCV library and NumPy for high-speed matrix calculations, ensuring the project remains a “traditional” computer vision solution without relying on pre-trained deep learning models.

[Back to Table of Contents](#)

8. References

References

- [1] S. S. A. Shah, M. A. Khalil, S. I. Shah, and U. S. Khan, “Ball Detection and Tracking Through Image Processing Using Embedded Systems,” in *Proceedings of the 2018 International Conference on Frontiers of Information Technology (FIT)*, Islamabad, Pakistan, 2018.
- [2] R. Jayanth, A. Manoj Kumar, et al., “Ball Detection and Tracking through Image Processing using Python,” *PRYS International Journal of Engineering Technology and Management Sciences*, vol. 7, no. 3, 2023.
- [3] G. Bradsky, “Computer Vision Face Tracking for Use in a Perceptual User Interface,” Intel Corporation, 1998.
- [4] Z. Zivkovic, “Improved Adaptive Gaussian Mixture Model for Background Subtraction,” in *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, 2004.
- [5] “An Efficient Ball Detection Framework for Cricket,” *IJCSI International Journal of Computer Science Issues*, vol. 7, no. 3, 2010.
- [6] “AI Cricket Umpire: Detection, Tracking and Trajectory Prediction,” *International Journal of Advanced Research in Computer and Communication Engineering (IJAR-CCE)*, vol. 14, no. 2, 2025.
- [7] S. V. Kamble and K. M. Bhurchandi, “A Comprehensive Review of Ball Detection Techniques in Sports,” *International Journal of Multimedia Information Retrieval*, 2019.
- [8] B. Chakraborty and S. Meher, “Real-Time Position Estimation and Tracking of a Basketball,” in *Proceedings of the IEEE International Conference on Signal Processing, Computing and Control (ISPCC)*, JUIT, India, 2012.
- [9] Q. Huang, S. Cox, X. Zhou, and L. Xie, “Detection of Ball Hits in a Tennis Game Using Audio and Visual Information,” in *Proceedings of the APSIPA Annual Summit and Conference*, 2012.
- [10] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.